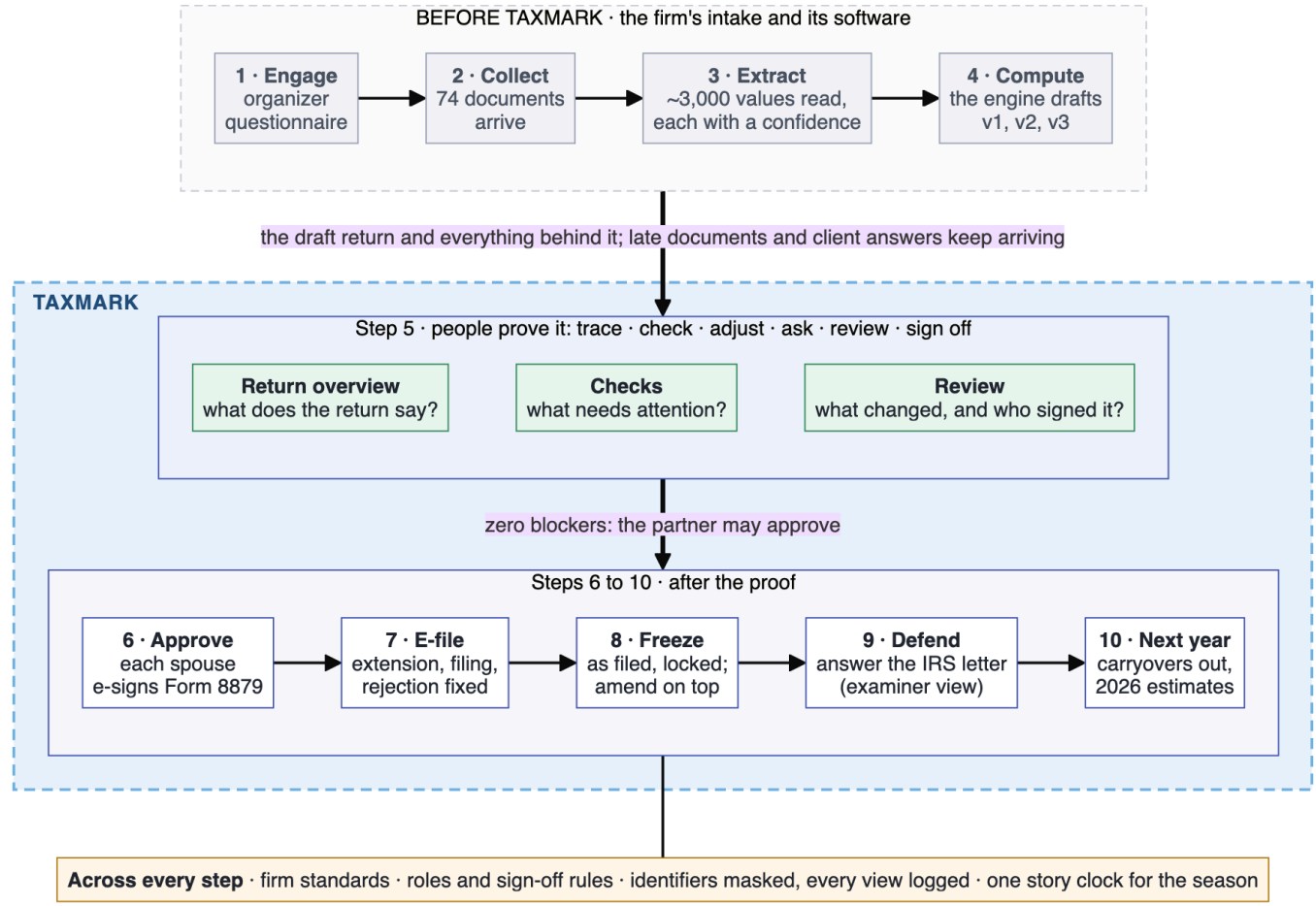
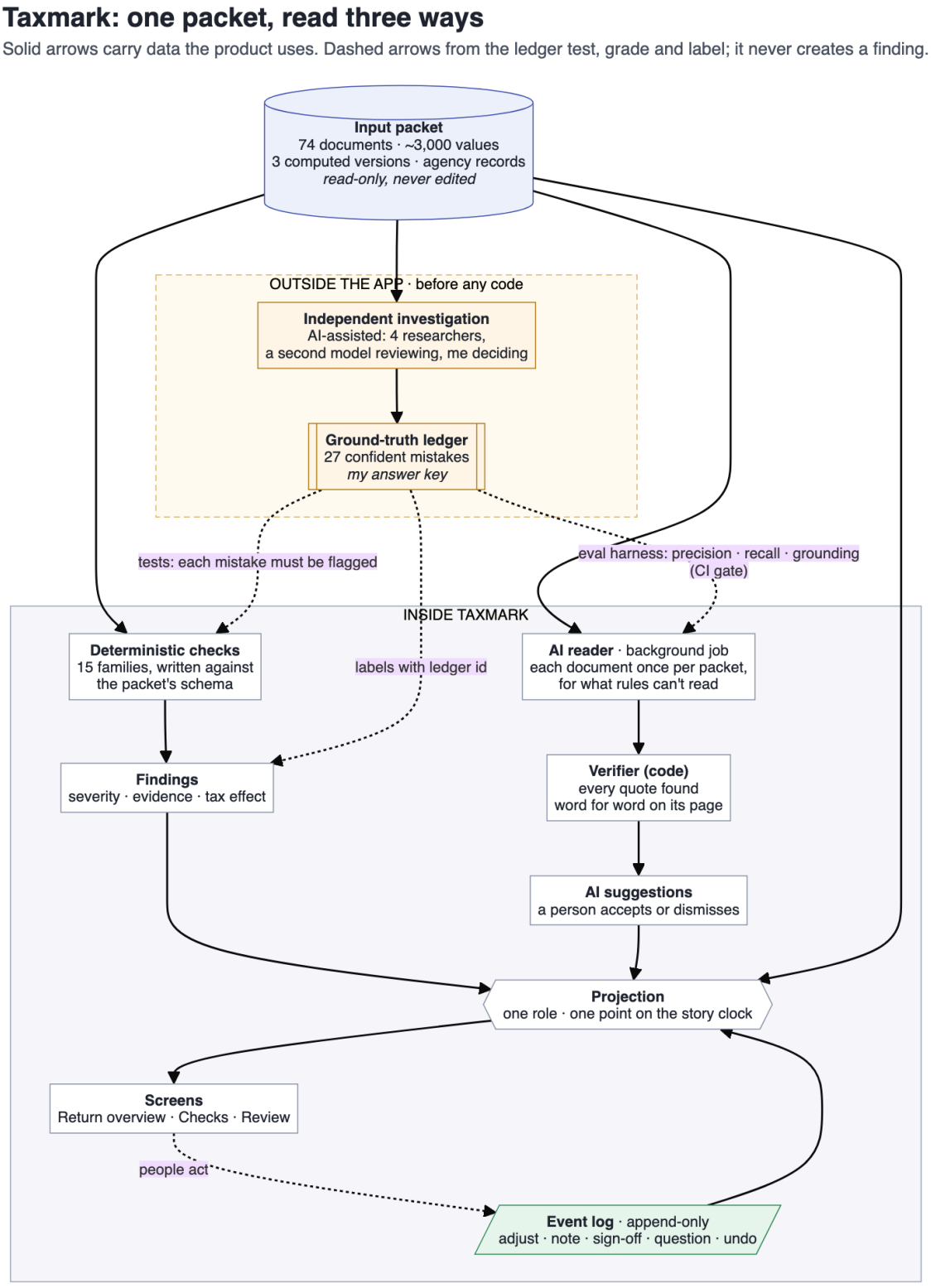
Taxmark brief: Appendix

A. Where Taxmark sits in the tax-filing pipeline



Steps 1 to 4 belong to the firm's intake and its software: the organizer, the documents, the extraction and the engine's draft. Taxmark picks the return up at step 5, where people prove it on three screens, each answering one question, and carries it through approval, e-filing, the frozen as-filed record, the IRS letter and into next year.

B. The data model



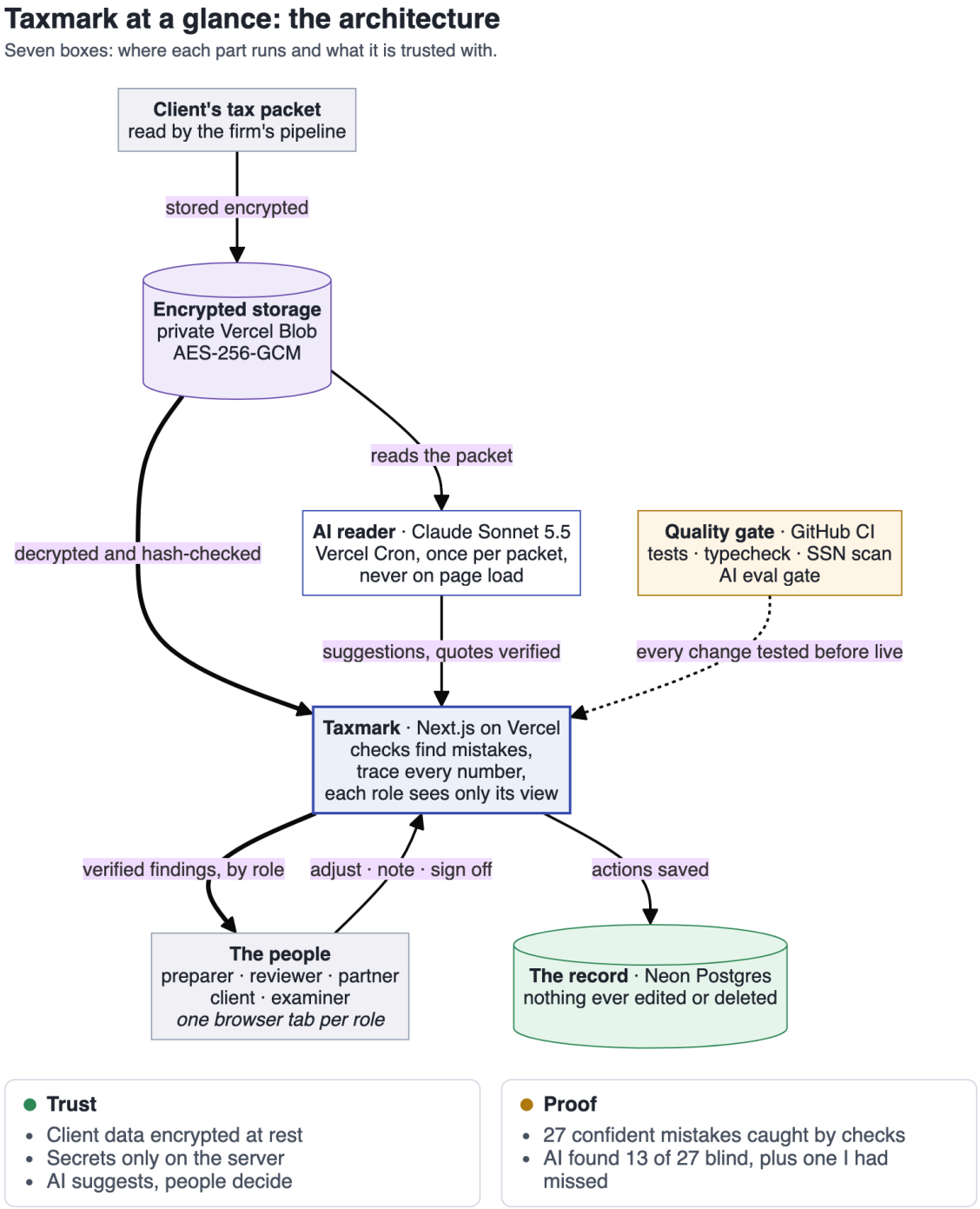
One packet, read three ways. The input packet was never split up or changed. The same packet was read by three readers, each with its own job, and that separation is what lets me trust the result:

- **The independent investigation (outside the app, before any code).** An AI-assisted hunt across the whole packet, with four AI researchers each taking one area, a second model reviewing their work, and me resolving anything in doubt, produced my ground-truth ledger: 27 distinct confident mistakes. This ledger is my answer key. It never creates a finding: the app only uses it to label a finding a check has already raised with its ledger id, and outside the app it tests the checks and grades the AI.
- **Deterministic checks (inside the app).** Each kind of mistake became a rule, 15 families in all, written against the packet's schema and never against a specific document, so the same checks would run on the next client's packet. There is a test for each of the 27 that fails if the app ever stops flagging it.
- **The AI reader (inside the app, for what rules can't read).** Some mistakes live in wording, like a client's answer or a note on a page, where rules are at their weakest. A background job runs an AI reader over every document once per packet, never while someone is browsing. Code then checks that every quote it gives is on its page, word for word, before a suggestion is shown, and a person accepts or dismisses each one.

Read-only underneath, everything on top. The packet and the AI reader's output stay read-only: never edited, only built on. Everything a person does sits on top of them:

- **Event log:** append-only. Adjustments, notes, sign-offs, questions, decisions, undo. Nothing is edited or deleted; an undo is a new event on top. The audit trail is the data, not a feature.
- **Projection:** for one role, at one point on the story clock: findings, blockers, sign-offs (fresh or stale), adjustments carried through the totals, masking.
- **Claim:** anything that can hold evidence and actions: a line, a document, a finding.

C. The architecture



Where each part runs. The packet is sealed with AES-256-GCM in a private Vercel Blob store and loaded by pointer, hash-checked, and refused if it does not match. A protected function on a daily Vercel Cron runs the AI reader (Claude Sonnet 5.5) with identifiers masked, and seals its output beside the packet. The Next.js app on Vercel runs the checks, the verifier and the projection on the server, enforces each role's policy and masks identifiers before anything reaches the browser. The event log lives in Neon Postgres, where the app only ever inserts and reads. GitHub Actions runs the unit tests (one per ledger mistake), a typecheck, an SSN scan and the AI eval gate, and the unit tests run again before every Vercel build.

D. How the AI reader was evaluated

Scored with a harness against the ledger (`docs/handin/ai-scorecard.md`):

Measure	Result
Grounding (every quote found on its page)	40 of 40 (100%)
Precision, same issue (judged by a second model)	25 of 40 (63%)
Recall, same issue (judged by a second model)	13 of 27 (48%)
Recall, automatic (any suggestion on the mistake's line)	17 of 27 (63%)
New real mistake not in the ledger	1

A test in CI fails the build if a suggestion is ungrounded or the automatic recall drops below 16 of 27.

E. The 27 confident mistakes

The full table, with the check that catches each one and its tax effect: `docs/handin/errors-found.md`.